BFA3 — Algo Trading — Problem 2

Build a Mini Matching Engine for a Central Limit Order Book

October 2025

Goal

Write a Python program that processes a list of incoming orders, maintains a Central Limit Order Book (CLOB), and produces a list of trade executions. Your engine must apply standard exchange rules: price–time priority, marketability, partial fills, and maker price.

Background

Electronic exchanges (equities, futures, crypto) centralize buy and sell interest in a **Central Limit Order Book (CLOB)** and match them with a **matching engine**. The book shows bids (buy limits) sorted best-to-worse and asks (sell limits) sorted best-to-worse; the matching engine continuously pairs compatible orders and emits trades.

Figure 1: New York Stock Exchange

Input file

The program reads a list of orders (ASCII/CSV-like), one order per line, comma-separated, no blank lines. Each line:

- `ts` *(int)*: timestamp in nanoseconds since Jan 1, 1970 (used for FIFO within a price level)
- `participant_id` *(str)*: market participant identifier
- `order_id` *(str)*: unique order id (tie-breaker if timestamps are equal)
- `symbol` *(str)*: e.g. `AAPL`
- `side` *(char)*: `B` for buy, `S` for sell
- `qty` *(int)*: strictly positive quantity
- `px` *(float)*: `0.0` means *market order*; otherwise this is the limit price

Input examples.

```
1527604196773077003,P1,o001,AAPL,B,300,101.20
1527604196773077004,P2,o002,AAPL,S,150,101.40
1527604196773077005,P3,o003,AAPL,B,200,0.0
```

Interpretation: Buy 300@101.20, then Sell 150@101.40, then a *market* Buy 200 (px=0.0).

Expected results

Return (or print) a dictionary with:

- `"executions"`: list of trade dicts, each including `exec_id, ts, symbol, price, qty, buy_order_id, buy_participant_id, sell_order_id, sell_participant_id`
- `"book"`: final book snapshot per symbol:

```
"book": {
  "AAPL": {
    "bids": [{"px": 101.2, "qty": 300, "orders": ["o001", ...]}, ...],  # best -> worse
    "asks": [{"px": 101.4, "qty": 150, "orders": ["o002", ...]}, ...]
  }
}
```

Output example (sketch).

```
{
 "executions": [
   {"exec_id":"e0001","ts":1527604196773077006,"symbol":"AAPL",
    "price":101.40,"qty":150,
    "buy_order_id":"o003","buy_participant_id":"P3",
    "sell_order_id":"o002","sell_participant_id":"P2"}
 ],
 "book": {"AAPL":{"bids":[{"px":101.20,"qty":300,"orders":["o001"]}],
                  "asks":[]}}
}
```

Explanation: the market buy matched the resting best ask 150@101.40 and removed it from the book; the 300@101.20 bid still rests.

Matching rules

All matching happens per symbol.

Price–time priority

- Bids: higher price first; within the same price, earlier `ts` first (if `ts` equal, lexicographic `order_id`)
- Asks: lower price first; within the same price, earlier `ts` first

Example: Two sells at 101.50 (ts=10 and ts=12). An incoming buy matches ts=10 first.

Marketability

Think: "Can it trade right now?"

- Buy is marketable if it is a *market order* (`px=0.0`) **or** the buy limit price is **at least** the current best ask (`px` $\geq$ best_ask).
- Sell is marketable if it is a *market order* (`px=0.0`) **or** the sell limit price is **at most** the current best bid (`px` $\leq$ best_bid).

Examples:

- Best ask = 101.40. Buy 50@101.40 is marketable; Buy 50@101.30 is not.
- Best bid = 101.20. Sell 20@101.10 is marketable; Sell 20@101.30 is not.
- A market order (`px=0.0`) is always marketable when opposite liquidity exists.

Trade price

Trades execute at the **resting order's price** (maker price). If a buy 100@102.00 hits a resting sell 100@101.60, the trade prints at 101.60.

Partial fills

Use what you can now, then keep going if the order is still marketable.

- If the incoming order quantity is larger than the resting order at the best price, consume it entirely and continue to the next best level as long as the order remains marketable.
- If the incoming order quantity is smaller, you partially fill the resting order and stop (the resting order keeps its remaining quantity).

Mini example: Best asks: 101.40 (qty 50), 101.50 (qty 70). Incoming market buy qty 80:

- Trade 50 @ 101.40 (level empty), remaining buy qty = 30
- Still marketable (next ask exists), trade 30 @ 101.50; remaining buy qty = 0

Symbols and isolation

Orders only match within the same `symbol`. Maintain separate books (bids/asks) per symbol.

Timestamps and tie-break

If two resting orders share the same price and timestamp, break the tie by lexicographic order of `order_id`.

Program API

Create a single file `engine.py` with:

```
def process_orders(orders: list[dict]) -> dict:
    """
    orders: list of dicts with keys:
      ts, participant_id, order_id, symbol, side, qty, px
    returns:
      {"executions": list[dict], "book": dict}
    """
```

Test and validation

Run the provided tests with:

```
pytest -q
```

Brief description of test cases

The tests check: (1) symbol isolation, (2) FIFO within the same price, (3) best-price beats earlier but worse price, (4) maker-price rule, (5) partial fills across multiple levels, (6) non-marketable orders rest, (7) independent books for multiple symbols, (8) symmetric behavior for market sells when liquidity exists, (9) tie-break using timestamp then order id, (10) book integrity after trades.

Hints

1. Keep, for each symbol, two maps: `price -> deque of resting orders` for bids and asks, plus two sorted price lists (bids descending, asks ascending). Clarity over speed.
2. For each incoming order: choose the opposite side; while it is marketable and quantity $>$ 0, consume from the best price level, FIFO within that level, emitting one execution per fill chunk.
3. If a limit order is not fully matched and becomes non-marketable, add its remaining quantity to the book at its limit price.
4. For ties: earlier `ts` first; if equal `ts`, lexicographic `order_id`.

A good read

An Introduction to Matching Engines (Databento, 2024). Entry-level guide with clear diagrams and simple explanations. URL: `https://medium.databento.com/an-introduction-to-matching-engines-a-gui`